

Program Development Specification (PDS)
Student Name: Alexander Taylor
Date: July 24, 2026
Project Name: Web-Based Budget Planner and Tracker

1. Program Purpose

- **What problem does this program solve?** It provides a centralized digital system to manage personal finances, organize expenses, and track financial habits over time.
- **Who will use this program?** Individuals, households, or small business owners looking to monitor their budgets and gain insights into their spending behavior.

2. Program Description

- **Describe in plain English what the program will do from start to finish:** The program will present the user with a web interface upon launch. The user can log daily expenses and income, categorize them, and add descriptions. These inputs will be sent to a backend server and saved securely in a database for long-term tracking over time. The web interface will retrieve this database information to display a real-time dashboard featuring the current balance, recent transactions, and spending summaries.

3. Inputs

- **What information must the user provide?**

Input	Data Type	Example
Transaction Type	String (or Enum)	"Income" or "Expense"
Amount	Float / Decimal	45.50
Category	String	"Groceries", "Rent", "Salary"
Date	Date	2026-07-24
Description	String	"Weekly grocery shopping"

4. Outputs

- **What information will the program display or produce?**

Output	Description
Current Balance	The total remaining funds after all tracked income and expenses are tallied.
Transaction History	A chronological table displaying past inputs retrieved from the database.
Category Summaries	Visual breakdowns (like charts) showing total spending per category over a specific time frame.

5. Processing

- **Describe the major calculations or decisions the program must perform.**
- **Processing Steps:**
 1. Receive and parse transaction data submitted through the web interface.
 2. Validate the input data to ensure correct formats (e.g., valid dates, positive amounts).
 3. Route the validated data to the database to insert a new transaction record.
 4. Query the database for all records within a requested date range.
 5. Calculate the total balance by summing all income and subtracting all expenses.
 6. Group expenses by category and sum the amounts for reporting.
 7. Send the processed data back to the web interface to render the dashboard.

6. Variables

- **List the variables you expect to use.**

Variable Name	Data Type	Purpose
transactionAmount	Float	Stores the numerical value of a single transaction.
transactionDate	Date	Stores the date the transaction occurred.
categoryList	Array of Strings	Holds the available categories for the user to choose from.
totalIncome	Float	Stores the aggregated sum of all income.
totalExpenses	Float	Stores the aggregated sum of all expenses.
netBalance	Float	Stores the calculated result of totalIncome minus totalExpenses.

7. Constants

- **List any constants that should be used.**

Constant Name	Value	Purpose
DATABASE_URL	String (Environment Variable)	Stores the connection string required to connect to the database.
MAX_DESC_LENGTH	Integer (255)	The maximum number of characters allowed in a transaction description.
CURRENCY_SYMBOL	String (\$)	Used for formatting monetary outputs on the web interface.

8. Validation Requirements

- **What input validation is required?**
- **Validation Rules:**

- **Amount:** Must be a positive numeric value greater than zero. Cannot be left blank.
- **Date:** Must be a valid calendar date. Cannot be left blank.
- **Category:** Must match one of the predefined categories or a user-created category. Cannot be left blank.
- **Description:** Optional, but if provided, must not exceed MAX_DESC_LENGTH characters.

9. Program Logic

- **Describe the decisions the program must make.**
- **Decision Rules:**
 - **IF** transaction type is "Income", **THEN** add the transactionAmount to the netBalance.
 - **IF** transaction type is "Expense", **THEN** subtract the transactionAmount from the netBalance.
 - **IF** a database query fails, **THEN** log the error and display a fallback error message on the web interface.

10. User Interface Design

- **Draw what the user will see.** (*Descriptive layout for the web interface*)
 - **Header:** Title "Budget Planner & Tracker" with a user profile icon.
 - **Sidebar Navigation:** Links for "Dashboard", "Add Transaction", "Reports", and "Settings".
 - **Main Content Area (Dashboard):**
 - Top Row: Three large summary cards displaying Total Income, Total Expenses, and Current Balance.
 - Middle Row: A pie chart showing Expenses by Category.
 - Bottom Row: A list/table showing the 5 most recent transactions (Date, Category, Description, Amount).

11. Is a menu required, and if so, what functionality should be provided?

- Yes, a navigation menu is required for the web interface.
- **Functionality:** It should allow users to switch between the main dashboard, the input form to add new transactions, a reporting page for historical charts (tracking over time), and a settings page to manage custom categories.

12. Special behaviors on startup or shutdown

- **Startup:** The backend server must establish a secure connection to the database before accepting web interface requests.
- **Shutdown:** The backend server must safely terminate the database connection to prevent data corruption.

13. File Requirements

- **Will the program read or write files?** Yes.
- **If yes, describe:** The program relies on a database to read and write all transaction records to ensure accurate tracking over time. It will read database rows to display data and write new rows when a user submits a form.

14. Error Handling

- **What errors could occur?**

Error	Program Response
Invalid Input Format (e.g., text in amount field)	Display an inline validation error on the web interface asking the user to enter a valid number.
Database Connection Timeout	Display a "Service Temporarily Unavailable" screen on the web interface and log the error for the administrator.
Missing Required Fields	Prevent form submission and highlight the missing fields in red.

15. Testing Plan

- Provide test data before coding.

Test #	Input	Expected Result
1	Type: Income, Amount: 1000.00, Cat: Salary	Record saved to database; Dashboard balance increases by \$1000.00.
2	Type: Expense, Amount: 50.00, Cat: Food	Record saved to database; Dashboard balance decreases by \$50.00.
3	Amount: -10.00	Validation fails; web interface shows "Amount must be greater than zero" error.
4	Navigate to "Reports" tab	Web interface retrieves historical database records and displays spending charts.